FA 4

Aldwin Benidict Montilla & Ben Alexander Libres

2026-09-10

GitHub 1: https://github.com/aldwinmontilla661-dot/Aldwin-Benidict-Montilla/tree/main/FA4

GitHub 2: https://github.com/benalexlibres/APM1110-LibresBen/tree/main/FA4

Item 2

A binary communication channel carries data as one of two sets of signals denoted by 0 and 1.

- A transmitted 0 is sometimes receives as a 1, and a transmitted 1 is sometimes received as a 0.
- A transmitted 0 is correctly received with probability 0.95.
- A transmitted 1 is correctly received with probability 0.75.
- 70% of all messages are transmitted as a 0.

If a signal is sent, determine the probability that:

(a). A 1 was received;

(b). A 1 was transmitted given that a 1 was received

Define the probabilities of the given data and events

- $P(T_0) = 0.70$ (0 is transmitted)
- $P(T_1) = 1 - P(T_0) = 0.30$ (1 is transmitted)
- $P(R_0|T_0) = 0.95$ (0 received given that 0 is transmitted)
- $P(R_1|T_0) = 1 - 0.95 = 0.05$ (1 is received given that 0 is transmitted)
- $P(R_1|T_1) = 0.75$ (1 is received given given that 1 transmitted)
- $P(R_0|T_1) = 1 - 0.75 = 0.25$ (0 is received given that 1 is transmitted)

Part (a): Probability that a 1 was received

In this type of conditional probability, we can use the **Law of Total Probability**. The total probability of receiving a 1 is the sum of **receiving a 1 when a 1 is transmitted** and **receiving a 1 when a 0 is transmitted.**

```r
p_T0 <- 0.70 # 0 is transmitted
p_T1 <- 0.30 # 1 is transmitted

p_R0_given_T0 <- 0.95 # 0 received given that 0 is transmitted
p_R1_given_T0 <- 0.05 # 1 is received given that 0 is transmitted
```

```r
p_R1_given_T1 <- 0.75 # 1 is received given given that 1 transmitted
p_R0_given_T1 <- 0.25 # 0 is received given that 1 is transmitted


# Solve for P(R1) (1 is received)
p_R1 <- (p_R1_given_T1 * p_T1) + (p_R1_given_T0 * p_T0)

cat("The probability that a 1 was received:",  p_R1)
```

```
## The probability that a 1 was received: 0.26
```

There is a **26%** chance that a **1 is received** if a signal was sent to a binary communication channel.

Part (b): Probability that a 1 was transmitted given that a 1 was received

In this type of problem, we can calculate the probability using Bayes' Theorem.

```r
p_T0 <- 0.70 # 0 is transmitted
p_T1 <- 0.30 # 1 is transmitted

p_R0_given_T0 <- 0.95 # 0 received given that 0 is transmitted
p_R1_given_T0 <- 0.05 # 1 is received given that 0 is transmitted

p_R1_given_T1 <- 0.75 # 1 is received given given that 1 transmitted
p_R0_given_T1 <- 0.25 # 0 is received given that 1 is transmitted


# Solve for P(R1_given_T1)

p_T1_given_R1 <- (p_T1 * p_R1_given_T1) / p_R1

cat("The probability that a 1 was transmitted given that a 1 was received P(T1 | R1):", p_T1_given_R1,
```

```
## The probability that a 1 was transmitted given that a 1 was received P(T1 | R1): 0.8653846 or 0.87
```

Applying Bayes' Theorem for the calculation of a problem involving posterior probability; in this instance, we are looking at the likelihood that a 1 was put out on the line given its receipt. This allows us to update our prior probability $P(T_1) = 0.30$ with the signal we have observed. And arrived at a probability of 0.87 (or 0.8653846).

Item 7

There are three employees working at an IT company: Jane, Amy, and Ava, doing 10%, 30%, and 60% of the programming, respectively. 8% of Jane's work, 5% of Amy's work, and just 1% of Ava's work is in error. What is the overall percentage of error? If a program is found with an error, who is the most likely person to have written it?

```r
people      <- c("Jane", "Amy", "Ava")
prior       <- c(0.10, 0.30, 0.60)   # proportion of total programming
error_rate  <- c(0.08, 0.05, 0.01)   # P(error | person)

# Sanity check: priors must sum to 1
stopifnot(abs(sum(prior) - 1) < 1e-9)

# Joint probabilities P(person AND error)
joint <- prior * error_rate

# Total probability of an error (law of total probability)
total_error <- sum(joint)

# Posterior probabilities P(person | error) via Bayes' Rule
posterior <- joint / total_error

# Sanity Check: posterior probabilities should sum to 1
stopifnot(abs(sum(posterior) - 1) < 1e-9)

# Identify most likely author
most_likely <- people[which.max(posterior)]

# --- Output ---
results <- data.frame(
    Person = people,
    Prior = prior,
    ErrorRate = error_rate,
    Joint = round(joint, 4),
    Posterior = round(posterior, 4))

print(results)
```

```
##   Person Prior ErrorRate Joint Posterior
## 1   Jane   0.1      0.08 0.008    0.2759
## 2    Amy   0.3      0.05 0.015    0.5172
## 3    Ava   0.6      0.01 0.006    0.2069
```

```r
cat(sprintf("\nOverall error rate: %.2f%%\n", total_error * 100))
```

```
##
## Overall error rate: 2.90%
```

```r
cat(sprintf("Most likely author of an error: %s (P = %.2f%%)\n",
            most_likely, max(posterior) * 100))
```

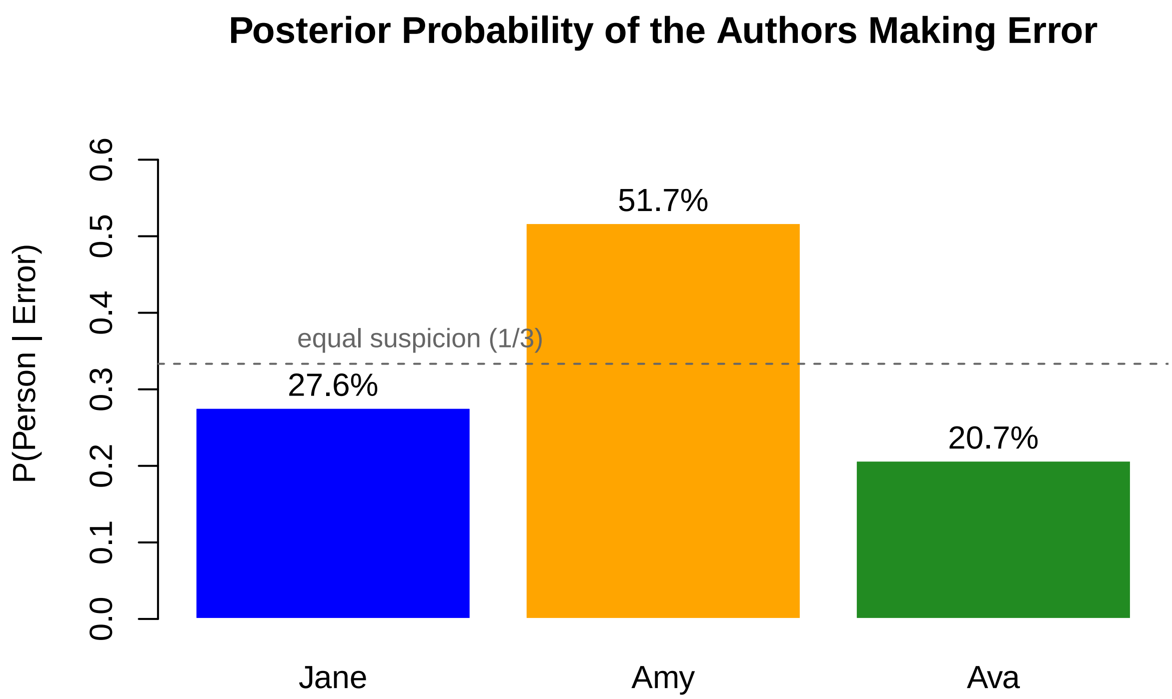
```
## Most likely author of an error: Amy (P = 51.72%)
```

BarPlot of the Posterior Probabilities

```r
bp <- barplot(posterior,
        names.arg = people,
        col = c("blue", "orange", "forestgreen"),
        ylim = c(0, max(posterior) + 0.15),
        ylab = "P(Person | Error)",
        main = "Posterior Probability of the Authors Making Error",
        border = "white")

# Adding value labels on top of each bar
text(x = bp, y = posterior + 0.03,
    labels = paste0(round(posterior * 100, 1), "%"))

# Reference line at 1/length(people) as an "equal suspicion" baseline
abline(h = 1/length(people), lty = 2, col = "gray40")
text(0.5, 1/length(people) + 0.03,
paste0("equal suspicion (1/", length(people), ")"),
        pos = 4, col = "gray40", cex = 0.8)
```



Summary

Across all three developers, the total prior probability of encountering an error in any given program is 1.90% (calculated using the Law of Total Probability: $0.10 \times 0.08 + 0.30 \times 0.05 + 0.60 \times 0.01 = 0.0190$). As a Breakdown: Jane has written 10% of the total code base, but due to her 8.0% error rate, she accounts for 42.11% of the overall errors ($P = \frac{0.008}{0.019}$). Amy has written 30% of the total code base with a 5.0% error rate, accounting for 39.47% of the overall errors ($P = \frac{0.015}{0.019}$). Ava has written 60% of the total code base with a 1.0% error rate, accounting for 18.42% of the overall errors ($P = \frac{0.006}{0.019}$). Jane is the most likely author of a program found with an error ($P \approx 42.11\%$).

Despite writing the smallest overall share of code (10%), Jane's disproportionately high individual error rate shifts the posterior distribution, making her the primary suspect when a bug is detected—exceeding both Amy (39.5%) and Ava (18.4%), as well as the random "equal suspicion" baseline of 33.3%).